

LocalAIAgent for Mac: design, features and improvement plan

Version 0.17.1 · October 2026 · Mac (Apple Silicon) only

LocalAIAgent is a personal AI agent that runs entirely on your Mac: - local language models through Ollama; - local memory in one SQLite file and one Markdown file; - Apple apps reached through EventKit, AppleScript and Accessibility.

Nothing leaves the Mac unless you approve that specific transfer. This document covers three things: - the current architecture and design; - the feature list as of 0.17.1; - every improvement opportunity found so far, under optimisation, trust and safety, security, and new features.

It covers the Mac only. Windows and Linux are left out on purpose.

Companion references in docs/: - DESIGN.md: the deep technical reference. - SECURITY_REVIEW.md: findings and their fixes. - TRUST_UX.md: the trust and controls analysis. - PRODUCT_ROADMAP.md: positioning against cloud agents.

1. Summary

What it is	A Muse/Instinct-class personal agent: chat, memory, calendar, mail, messages, files, web, Mac apps, voice, proactive nudges, all on-device.
Who it's for	Mac users who want an assistant that acts for them without moving their life into a vendor's cloud.
How it runs	One Python process (localagent), launched as LocalAIAgent .app so macOS permissions attach to the app. It talks to Ollama on 127.0.0.1 and serves a web UI on 127.0.0.1:8765.
Models	qwen3:4b (chat and tools), qwen3:1.7b (fast decisions), all-minilm (embeddings), Whisper on MLX (speech), macOS say (voice).
Cost	\$0 per token. An Anthropic key is optional, for "think harder" (each request needs your approval).
State	0.17.1. 266 automated tests, with CI on macOS, Ubuntu and Windows. Releases are verified with browser checks and upgrade tests.
Top 3 next steps	(1) Set the model's context window explicitly: today, long pages and emails silently push the safety instructions out. (2) Ship as a signed, notarised app with a menu-bar presence and actionable notifications. (3) Use Apple's on-device frameworks where they beat Ollama and Whisper.

2. Principles

Principle	In practice
Local-first	Models, memory, audio and screen text never leave the Mac. The UI listens on 127.0.0.1 only.
Decide first, generate second	A ~40 ms decision layer classifies every input (intent, memorable?, complexity, should-nudge, urgency) before a large model runs.

Principle	In practice
Policy over model	The model only <i>proposes</i> actions. A deterministic policy engine decides from each tool's risk tier. The model can never approve itself.
Visible and reversible	Every decision shows probabilities. Every action, approval, refusal and setting change goes into a hash-chained audit log. Memory is a plain file. Deletes go to the Trash.
Explicit egress	Anything that leaves the Mac (email, iMessage, web query, cloud question) is listed in Trust → Activity, with how it was approved.
Fail closed	Without a sandbox, skills don't run. The phone line is read-only. After untrusted content is read, standing permissions stop applying.

3. Architecture

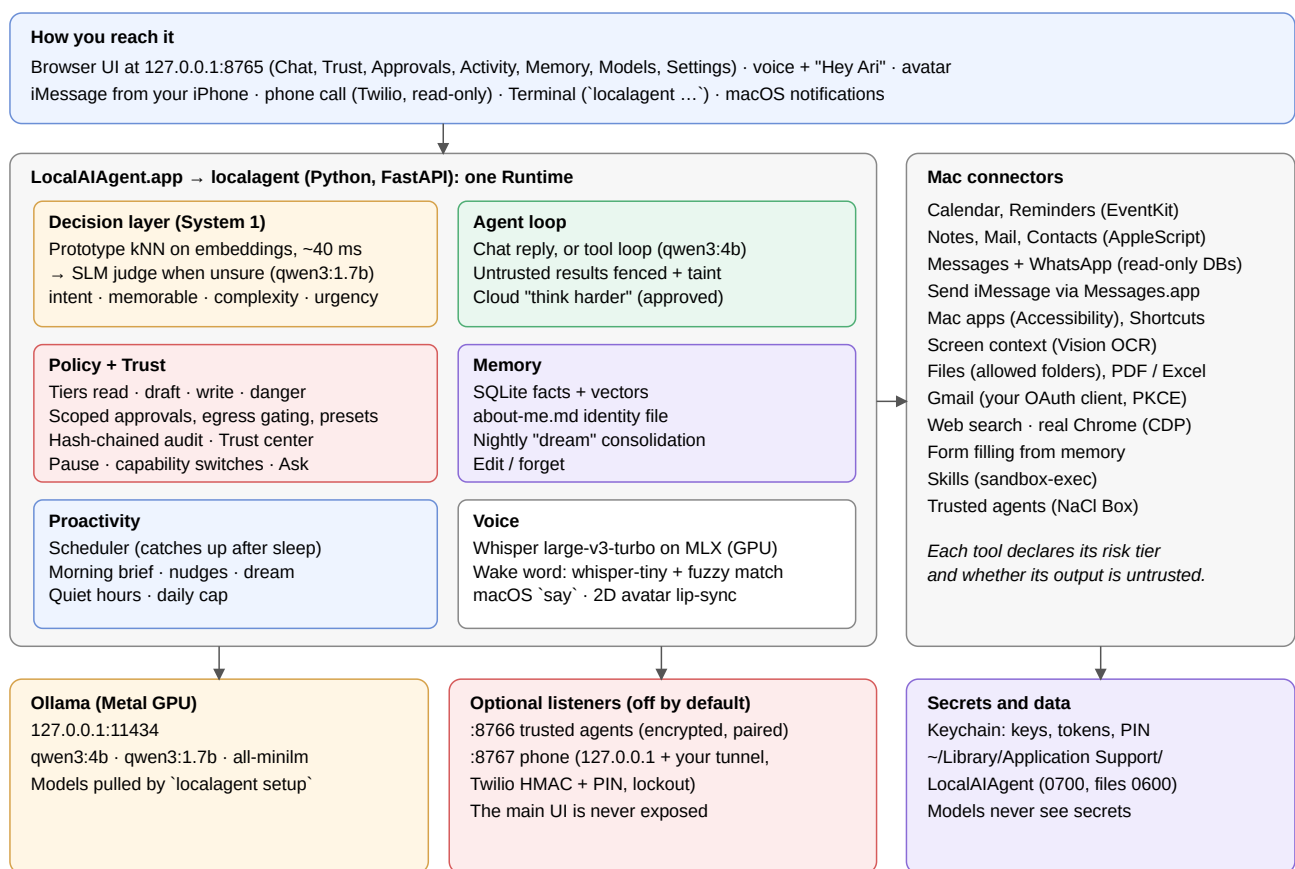


Figure 1. Components on the Mac. One agent process; anything reachable from outside is a separate listener, off by default.

3.1 Process model on macOS

Piece	How it works	Why
LocalAIApp	An applet built on your Mac with Apple's <code>osacompile</code> , ad-hoc signed, in <code>~/Applications</code> . It runs the agent with <code>--no-open</code> . <code>localagent</code> app install launches it once to check that macOS accepts it.	macOS permissions (Calendar, Automation, Full Disk Access, Accessibility, Screen Recording) belong to the app, not to Terminal or Python. If macOS refuses the app (error -10669), the agent falls back to running from Terminal.
Autostart	A LaunchAgent in <code>~/Library/LaunchAgents</code> opens the app at login with <code>/usr/bin/open -g -a</code> .	It starts in the background, with the same identity as when you launch it by hand.

Piece	How it works	Why
Sign-in	A per-install secret, sent either as an HttpOnly, SameSite=Strict cookie or as a Bearer header. <code>localagent</code> open gets a one-time code (single use, 60 s) and opens <code>/auth?c=...</code> . The secret is never in a URL.	Other local apps and web pages can't drive the agent. Browser history holds nothing reusable.
Request guard	Host allowlist (blocks DNS rebinding); same-origin check on writes; confirmation (<code>X-Confirm</code>) for sensitive settings; CSP and nosniff headers.	Defence against cross-site and rebinding attacks on a localhost service.
Background work	A scheduler loop for the brief, checks and dream; an iMessage poller with backoff. Trust checks, exports and the overview run off the event loop.	One slow job never freezes chat or voice.

3.2 A message, end to end

1. **Input** arrives from the UI, iMessage, a phone call, the wake word or a scheduled job.
2. **Decide.** The text is embedded and scored against labelled prototypes (~40 ms), giving intent, memorable?, complexity 1–5, should-nudge and urgency. Below the confidence threshold, the 1.7B judge re-decides with JSON-schema output. Your corrections become new prototypes immediately.
3. **Privacy questions** ("what did you send?", "who can control you?") are answered from the agent's own records, not by the model.
4. **Recall** memories (vector search plus `about-me.md`).
5. **Route.** Small talk goes to the fast model. Tasks go to the tool loop on the 4B model, with only the relevant tools. Complexity 5 or "think harder" may offer the cloud model, behind an approval.
6. **Act.** For every proposed call, the policy engine applies the tool's tier for those arguments, capped by the trust preset (Observer, Assistant or Agent) and by Pause:
 - read and draft run;
 - write needs approval or a standing, target-scoped grant;
 - danger needs fresh approval every time.

After anything written by others is read (mail, chats, web, screen, peers), the run is tainted. Sends then ignore standing grants. New links, or words not from your request, need approval to leave the Mac. 7.

Answer streams to the UI with tool chips, approval cards (full payload and "why it asks") and decision chips. 8. **Remember and audit.** Memorable facts are extracted and deduplicated. Everything is logged in the hash chain.

3.3 Code map

Area	Modules
HTTP, UI, sign-in	<code>server.py</code> , <code>security.py</code> , <code>ui/</code> (vanilla JS, no build step, SSE)
Wiring and app	<code>runtime.py</code> , <code>config.py</code> , <code>cli.py</code> , <code>macapp.py</code> , <code>launcher.py</code> , <code>doctor.py</code>
Decisions	<code>decide/</code> (prototype, SLM judge, router, typed questions)
Agent	<code>agent/chat.py</code> , <code>agent/actions.py</code> , <code>agent/cloud.py</code>
Policy and trust	<code>policy/engine.py</code> , <code>safety/egress.py</code> , <code>safety/injection.py</code> , <code>safety/pii.py</code> , <code>trust.py</code> , <code>trust_ask.py</code> , <code>vault.py</code>
Memory	<code>memory/store.py</code> , <code>memory/identity.py</code>
Proactivity	<code>proactive/</code> (scheduler, jobs, nudges)

Area	Modules
Connectors	<code>connectors/</code> (one file per integration)
Channels	<code>channels/imessage.py</code> , <code>channels/phone.py</code> , <code>connectors/peers_server.py</code>
Voice	<code>voice/</code> (MLX Whisper, wake word, <code>say</code>)

4. Design

4.1 Models

Role	Default	Notes
Chat and tools	<code>qwen3:4b</code>	Native tool calling; fits 16 GB with headroom
Fast and judge	<code>qwen3:1.7b</code>	Decision escalation, summaries, Ask wording
Embeddings	<code>all-minilm</code>	384-dim, English-centred; due for replacement (\$6.1)
Speech to text	Whisper large-v3-turbo on MLX	On the GPU; audio never leaves the Mac
Wake word	Whisper tiny.en on MLX	Short bursts only; phrase matched with fuzzy, sound-alike rules
Voice	macOS <code>say</code>	Any installed voice
Cloud (optional)	Anthropic, with your own key	Each question needs approval; memories excluded unless enabled

4.2 Decision layer

Typed questions (choice, yes/no, score) get probabilities from a prototype nearest-neighbour classifier, about 40 ms per input. When it isn't confident enough, the 1.7B judge decides instead. The UI shows every decision's probabilities, and one click corrects it. An eval harness reports accuracy, F1, calibration (ECE) and latency.

4.3 Actions and policy

- **Four tiers.** Each tool declares its tier, sometimes per call: clicking "Pay" is danger, while clicking "Next" is draft.
- **Approvals.** Six scopes: once, task, session, 1 h, 24 h, always. "Always" expires after 30 days. Grants are keyed by target (recipient, chat, site).
- **Egress gating.** It works from data flow, not from a pattern list: after untrusted reads, anything that would leave the Mac needs approval.
- **Presets and Pause.**
 - Observer: look only.
 - Assistant: also drafts.
 - Agent: also acts, with approval.
 - Pause: read-only, even over iMessage ("pause" / "resume").

4.4 Trust & Transparency center (0.17)

- **A status line:** "All good" or "N need your attention", with fixes, Pause and the preset.
- **Four sub-tabs:**
 - Overview: Ask about privacy and security, plus a weekly summary.
 - Capabilities: 20 switches. Each shows what it reads, what can leave, the risk, the safeguards and the macOS permission, with a link to that System Settings page.
 - Activity: what left this Mac, and standing permissions.

- Verify & export:
- automated checks on the log;
- a redacted zip with SHA-256 manifest and a ready-made prompt, for an outside reviewer such as Claude or ChatGPT.
- **Safety checks:**
 - disk encryption (FileVault);
 - models running locally;
 - skills sandboxed;
 - app identity;
 - phone PIN and PIN lockout;
 - friends' agents verified;
 - cloud excludes memories;
 - no permanent grants;
 - activity log intact.

4.5 Memory

- **Storage:** facts with embeddings in SQLite, plus a human-readable `about-me.md` identity file. Memory can be edited and forgotten in the UI.
- **Recall:** the top-k by cosine similarity, on every message.
- **Nightly "dream":** merges, deduplicates and retires stale facts, when the Mac is awake.

4.6 Proactivity

- **Scheduler:** survives sleep, catching up once on wake.
- **Morning brief:** 08:00 by default.
- **Nudges:** for events, reminders, and unreplied mail and chats. Gated by `should-nudge × urgency`, with quiet hours (22:00–07:00 by default) and a daily cap.
- **iMessage forwarding:** optional, of nudges and approvals ("yes" approves).

4.7 Voice

- **Push-to-talk and conversation mode:** on-device Whisper.
- **Wake word:** "Hey Ari" works while the page is open, after one click (browsers keep audio paused until then).
 - The matcher ignores Whisper's sound labels and filler words.
 - Near misses show "Heard "..."" for a moment.
- **2D avatar:** lip-synced to the voice.

4.8 macOS permissions used

Permission (System Settings → Privacy & Security)	Used by	Asked when
Calendars, Reminders	EventKit connectors	First calendar or reminder request
Contacts	Contacts look-up, name redaction in exports	First contact request
Automation (Calendar, Notes, Mail, Messages, Contacts)	AppleScript connectors, sending iMessage	First use of each app
Full Disk Access	Reading Messages and WhatsApp databases (read-only copies)	When you turn Messages on

Permission (System Settings → Privacy & Security)	Used by	Asked when
Accessibility	Mac apps (pressing buttons, reading windows)	When you turn Mac apps on
Screen Recording	Screen context (on-device OCR)	When you turn Screen context on
Microphone (in the browser)	Voice and wake word	First use

4.9 Data and secrets

- **Data folder:** `~/Library/Application Support/LocalAI/Agent`, reset to 0700 (files 0600) at every start. It holds the SQLite database (messages, memories, decisions, audit, nudges, peers), `about-me.md`, `server.log` and the browser profile.
- **Keychain:** Gmail tokens, the Anthropic key, Twilio token, phone PIN, and agent and peer keys. Models only ever see "connected / not connected".
- **Screen text:** kept for 120 minutes by default (5 min to 24 h). Private apps are skipped and screenshots are deleted at once.

5. Feature list (Mac, 0.17.1)

✅ shipped · 🟡 partial · ❌ not yet

Area	Feature	Status	Notes
Core	Streaming chat, persona (name, tone), model manager	✅	Pull, unload and switch models in the UI
	Decision layer with visible probabilities and one-click corrections	✅	Eval harness included
	"Think harder" with Claude	✅	Your key; approval per question
Trust	Trust center: status, capabilities, presets, pause, activity, checks, export	✅	0.16–0.17
	Ask about privacy and security (Trust tab and chat)	✅	Answers from records
	Approvals with full payload and reasons; scoped grants; audit log	✅	
Apple apps	Calendar, Reminders, Notes, Mail, Contacts	✅	EventKit and AppleScript
	Mac apps via Accessibility; run Shortcuts	✅	Pressing buttons asks first
	Screen context (on-device OCR)	✅	Opt-in, high risk, short retention
Messages	Read iMessage/SMS and WhatsApp; "waiting on you" threads	✅	Read-only copies
	Reply by iMessage (approved); WhatsApp reply prepared for you	✅	
Mail	Mail.app and Gmail: search, read, follow-ups, drafts, send	✅	Gmail through your own OAuth client
Web	Search plus reading the source page	✅	Page text capped at 6,000 characters
	Real Chrome automation with a visible click overlay	✅	Submit and pay always ask

Area	Feature	Status	Notes
	Form filling from memory	✓	Never submits for you
	Checkout with payment	●	You complete the payment
Files	Search, list, move, trash, open; create PDF/Excel	✓	Allowed folders only; programs never opened
Proactive	Morning brief, nudges, nightly dream	✓	Quiet hours, daily cap
Channels	iMessage channel (text the agent, approve with "yes")	✓	Note-to-self or second Apple ID
	Phone line (Twilio)	✓	Read and draft only; PIN on every call; lockout after 5 wrong PINs
	Native iPhone app, Apple Watch	✗	iMessage and phone instead
Voice	Push-to-talk, conversation mode, "Hey Ari", avatar	✓	On-device
Agents	Trusted agents (free/busy, messages, questions)	✓	E2E encrypted, emoji safety code
	Concurrent sub-agents	●	One local model at a time
Extensibility	Custom skills (SKILL.md plus sandboxed scripts)	✓	No keychain access from the sandbox
App	LocalAIAgent.app, autostart, doctor, one-command upgrade	✓	Ad-hoc signed; not notarised
	Menu-bar app, actionable notifications, Shortcuts actions	✗	See §6.4

6. Improvement opportunities

Each item has an effort estimate (S < 1 day, M a few days, L a week or more) and a priority (P0 now, P1 next, P2 later).

6.1 Optimisation: speed, memory and long content

#	Opportunity	Why	Effort	Priority
O1	Set the context window (<code>num_ctx</code>) on every model call , sized to the input and capped by installed RAM	Nothing sets it today, so Ollama uses its default of a few thousand tokens and silently drops the <i>start</i> of long inputs: the system prompt and its safety rules. Pages (6,000 chars, kept for up to 5 tool steps) and emails (up to 20,000 chars) exceed it.	S	P0
O2	Keep models loaded (<code>keep_alive</code> : fast model always, chat model 30 min) and preload both at start	Models unload after 5 idle minutes, so the next message pays a cold load of several seconds. Today's warm-up only prepares embeddings.	S	P0
O3	Reorder the prompt so it can be cached : persona, rules and tool definitions first; history; then time and memories last	The second line of the system prompt is the time to the minute, followed by per-turn memories. That defeats Ollama's prompt-prefix cache, so instructions, tools and history are reprocessed on every turn.	S	P0

#	Opportunity	Why	Effort	Priority
O4	Ollama memory flags: OLLAMA_FLASH_ATTENTION=1 and OLLAMA_KV_CACHE_TYPE=q8_0, set by the installer	Roughly halves the memory a long context needs, which makes O1 affordable on 16 GB Macs	S	P1
O5	Timing telemetry: log Ollama's prompt and generation timings per call, and show them in Models	Measure before changing engines	S	P1
O6	Run independent decision calls in parallel in the router	Cuts latency on escalated decisions	S	P1
O7	Better page extraction: read schema.org product data first (title, price, rating, stock), then reader-mode text as Markdown; extractors for Amazon and a few big sites	Product pages are mostly menus and ads; structured data is exact and small	M	P1
O8	Keep pages out of the conversation: store the full page per task and add <code>page_search(query)</code> / <code>page_read(section)</code> tools, with chunks picked by embedding	Lets the agent read long pages and review lists without carrying 6,000 characters through every step	M	P1
O9	Compact old tool results after each step (keep a summary)	Keeps multi-step tasks inside the window	S	P1
O10	Split very long reads across calls (reviews, long articles): the fast model pulls what matters from each chunk, the chat model combines	Long documents on small models	M	P2
O11	Better embeddings (nomic-embed-text, or a Qwen3 or Gemma embedding model) plus keyword search with SQLite's full-text index, merged with the vector results	<code>all-minilm</code> is English-centred and reads only ~256 tokens of each text; names and order numbers need exact matching	M	P1
O12	Cache the memory vector matrix in RAM; update it on add, edit and forget	Today every message re-reads every embedding from SQLite	S	P2
O13	Rank memories by similarity × recency × use; consolidate duplicates during the dream	Better recall, smaller prompts	M	P2
O14	Summarise old conversation instead of sending 20 raw messages	Shorter prompts, faster replies	S	P2
O15	Apple's on-device model (Foundation Models framework, macOS 26) as an option for the fast/decision role	No download, tuned for Apple silicon, supports guided generation and tool calling. Needs a small Swift helper.	M	P2
O16	MLX inference backend (<code>mlx-lm</code>) behind the same client, chosen in Settings	Often faster than llama.cpp on Apple Silicon; speculative decoding (a tiny draft model) can speed up long replies further. Benchmark before switching. vLLM is not a fit (built for NVIDIA servers).	L	P2
O17	Apple speech (SpeechAnalyzer, macOS 26) as an alternative to Whisper for dictation and the wake word	Streaming, low-latency, on-device, no model download	M	P2
O18	Curated model choices by role , with capability checks through Ollama's <code>/api/show</code> (tools, vision) and <code>localagent eval --model X</code>	Safe way to offer Gemma 3 (long context, vision: good for reading pages and screenshots), Llama, Phi or larger Qwen3. Gemma 3 suits reading rather than acting unless it advertises tool support.	M	P1

6.2 Trust and safety

#	Opportunity	Why	Effort	Priority
T1	Never let instructions fall out of the window (O1), and refuse or summarise inputs that would	The safety rules and injection fence instructions must always be in front of the model	S	P0
T2	Per-channel tool lists (for example, phone: calendar and reminders only; iMessage: no mail bodies)	A phone call's speech goes through Twilio's cloud recogniser (review P2-10)	S	P1
T3	Anchor the audit chain in the Keychain (latest hash and count, plus an HMAC per row) and verify at start	Someone who can write the file can rewrite the chain and recompute every hash; deleting the tail goes unnoticed (P2-2)	M	P1
T4	Approve from a notification or your Watch: actionable macOS notifications for approvals, with the full payload one click away	Approvals shouldn't need the browser tab open	M	P1
T5	"What it saw" per capability: Trust shows the last items each connector read (titles only), not just what left	Completes the "what did you do with my data" picture	M	P2
T6	Quarantined reader for untrusted content: a separate model call that only extracts facts from untrusted text, so the acting model never sees raw instructions from others	Defence in depth beyond fencing, tainting and egress gating	L	P2
T7	Retention controls for chat history, audit and nudges (with exports kept), plus "forget everything about "	Users can bound what is kept	M	P2
T8	Focus-aware nudges: follow macOS Focus modes, not only fixed quiet hours	Fewer interruptions, more trust in proactivity	S	P2
T9	Weekly trust digest (one notification): what left, new grants, failing checks	Keeps the user informed without opening the tab	S	P2
T10	Nudge if Trust has needed attention for days , for example FileVault off or an unverified friend	Status that nobody sees doesn't help	S	P2

6.3 Security

#	Opportunity	Why	Effort	Priority
S1	Developer ID signing, notarisation and the hardened runtime for a real LocalAIApplet.app bundle (embedded Python), replacing the ad-hoc-signed applet	Gatekeeper trust; stable code identity for permissions; prerequisite for S2, S3 and §6.4 features	L	P1
S2	Keychain items limited to the signed app (access control by code-signing requirement)	Today any process running the same Python could ask for the agent's keys outside the sandbox	M	P1
S3	Signed auto-updates (for example Sparkle, with EdDSA signatures)	Upgrades without a manual wheel; no tampered packages	M	P1
S4	Encrypt the database (SQLCipher) with a key in the Keychain	Protects data in backups and synced folders, and on Macs without FileVault	M	P2
S5	Deny-by-default sandbox profile for skills: allow only the interpreter, the skill folder and declared network	The current profile allows by default, then denies the home folder, app automation and keychain daemons	M	P1
S6	Split privileged connectors into separate helpers (Full Disk Access for Messages, Accessibility, Screen Recording), each with only its own permission	A bug in one connector can't use the others' permissions	L	P2

#	Opportunity	Why	Effort	Priority
S7	Chrome over --remote-debugging-pipe instead of a local port	Removes a local attack surface on the agent's browser profile (P1-8 residual)	M	P2
S8	Pin the supply chain : hashed dependency lock in the wheel; models pulled by digest	Install and upgrade integrity (P2-6)	S	P1
S9	Narrower Gmail scope : read-only first, compose/send only when sending is turned on	Least privilege (P2-7)	S	P2
S10	Request size and rate limits on chat and audio uploads	Local denial of service (P2-8)	S	P2
S11	Redact personal text from server.log , create it 0600, and rotate it	Logs can hold tool output (P2-9)	S	P1
S12	Trusted-agent listener off public networks : default to the Tailscale or home-network interface, and warn on café Wi-Fi	P2-4	S	P2

6.4 Additional features (Mac)

#	Feature	What it gives	Effort	Priority
F1	Menu-bar app : status dot, Pause, quick ask, pending approvals, open the full UI	The agent feels like part of the Mac, not a browser tab	M	P1
F2	Global hotkey and floating ask panel (Spotlight-style)	Ask from anywhere in one keystroke	M	P1
F3	App Intents : "Ask Ari", "Add to Ari's memory", "Ari: brief me" as Shortcuts actions, available to Siri and Spotlight	Native automation and voice through Siri	M	P1
F4	Finder Quick Actions and a Share extension : summarise this PDF, file this receipt, remember this page	Acts on what you are looking at	M	P2
F5	Safari support for web tasks (alongside Chrome)	Many Mac users live in Safari	L	P2
F6	Wake for the morning brief (pmset scheduled wake) and run during Power Nap	The brief is ready even if the Mac slept	S	P2
F7	Photos search with on-device Vision: "find the photo of the receipt from March"	New, private capability	M	P2
F8	Mail rules and smart follow-ups : "remind me if nobody replies in 3 days"	Proactive mail without cloud	M	P2
F9	Scheduling assistant : find slots across your calendars and a friend's agent, then draft the invite	Uses existing trusted-agent free/busy	M	P2
F10	iPhone companion : a tiny app over the iMessage/peer channel, showing approvals and the brief	Closes the "no mobile app" gap without a cloud	L	P2
F11	Always-listening wake word without the browser (a native audio helper)	"Hey Ari" works with the UI closed	M	P2
F12	Multiple Mac users : one agent per macOS account, clearly separated	Shared family Macs	M	P2

7. Recommended roadmap

Release	Contents	Outcome
0.18 (now)	O1–O3, T1, O5, O6, S11	Long pages and emails are safe; replies faster with no cold starts; timings visible
0.19	O7–O9, O11, O18, T2, S8	Reads Amazon and other long pages well; better recall; tested model choices (Gemma 3 for reading)
0.20	S1–S3, F1–F3, T4	A signed, notarised Mac app with menu bar, hotkey, Shortcuts/Siri and approvals from notifications
0.21+	O15–O17, S4–S7, T3, T5–T7, F4–F12	Apple on-device frameworks, stronger isolation, richer Mac integration

8. Known limits (Mac)

- **Reasoning:** 4B local models trail frontier models on long multi-step plans. The router, short tool chains, skills and optional cloud help.
- **Availability:** the agent stops when the Mac sleeps (F6 helps for the brief).
- **Wake word:** works only while the UI page is open, and after one click per page load (browser audio rules). F11 removes this.
- **Commerce:** the agent stops before payment, by design.
- **Mobile:** no native iPhone app. iMessage and the phone line cover the basics.
- **App identity:** the ad-hoc-signed applet works, but some macOS versions refuse it; the Terminal fallback then holds the permissions. S1 resolves this.